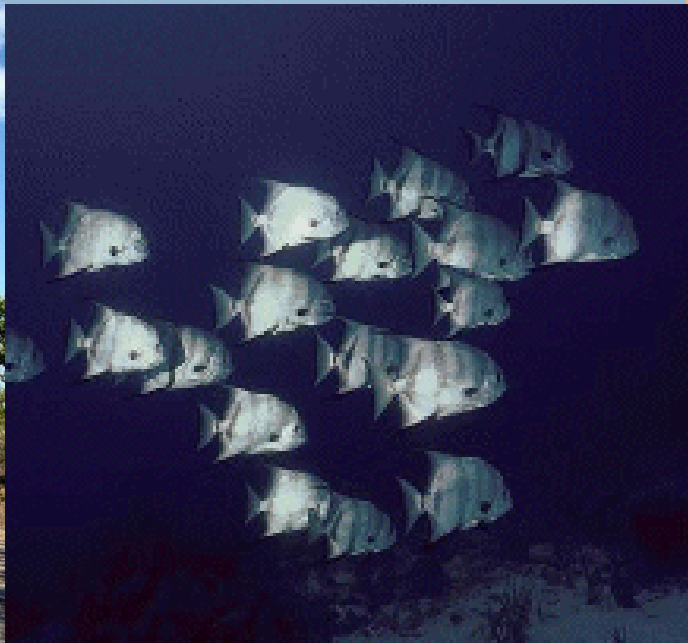




Malvina Reynolds:  
Some other eyes  
will look around,  
and find the things  
I've never found.

# Swarm Intelligence

## Lecture 7



- ❖ **What is Swarm Intelligence (SI)?**
- ❖ **Communication**
- ❖ **Simulate SI for Search**
  - Ant Colony Optimization (ACO)
  - Particle Swarm Optimization (PSO)
- ❖ **Supplement: Quantum Computing**

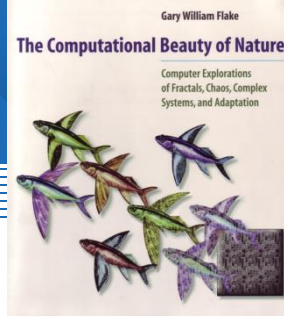


# Part I : What is Swarm Intelligence?

**Kevin Kelly: “*Dumb parts, properly connected into a swarm, yield smart results.*”**



# The Computational Beauty of Nature

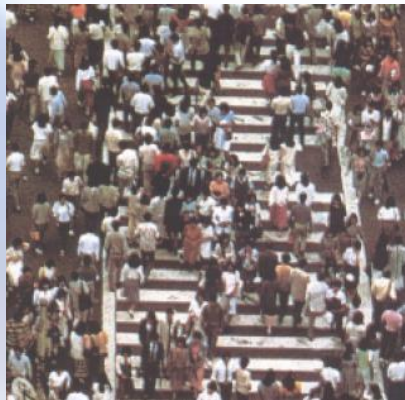
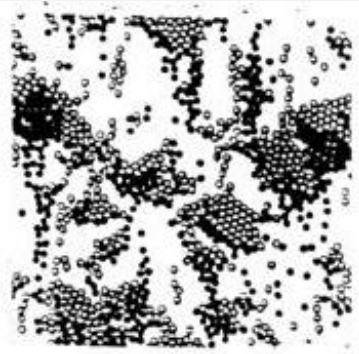
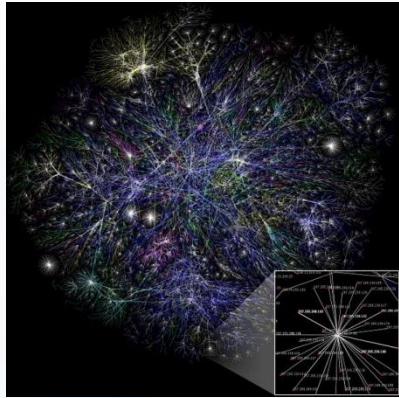


- ❖ Some social systems in Nature can present an intelligent collective behavior although they are composed by simple individuals.
- ❖ The intelligent solutions to problems naturally emerge from the self-organization and communication of these individuals.
- ❖ These systems provide important techniques that can be used in the development of artificial intelligent systems.





# Examples of Collective Behavior in Nature and Society



Which can be treated as Multi-Agent System



# Emergence

- ❖ Goldstein: "The arising of novel and coherent structures, patterns and properties during the process of self-organization in complex systems."
- ❖ Murray Gell-Mann: "Superficial complexity that arises from a deep simplicity"
- ❖ Bottom-up behavior: Simple agents following simple rules generate complex structures/behaviors. Agents don't follow orders from a leader.



A termite "cathedral" mound produced by a termite colony: a classic example of emergence in nature.

# Biological motivation: Insect Societies

- ❖ Colonies of social insects can achieve flexible, reliable, intelligent, complex system level performance from insect elements which are stereotyped, unreliable, unintelligent, and simple.
- ❖ Insects follow simple rules, use simple local communication (scent trails, sound, touch) with low computational demands.
- ❖ Global structure (e.g. nest) reliably emerges from the unreliable actions of many.





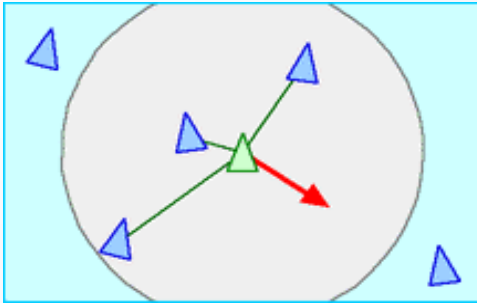
# Biological motivation: Flocks, Herds and Schools

- ❖ In the late 80's Craig Reynolds created a simple model of animal motion that he called Boids.
- ❖ It's generates very realistic motion for movement from three simple rules which define a boid's steering behaviour.
- ❖ This model, and its variations, has been used to drive animations of birds, insects, people, fish, antelope, etc. in films (e.g., Batman Returns, Lion King)



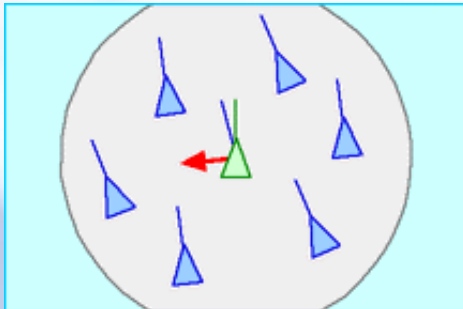


# Boid rules



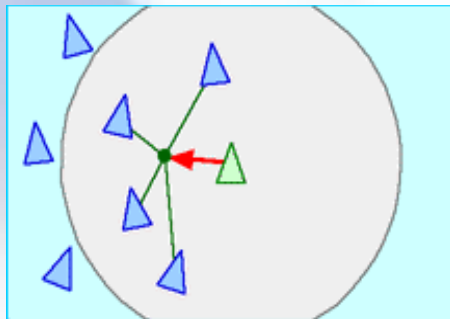
**Separation:** steer to avoid crowding local flockmates

- A fundamental rule that has priority over the others
- Also useful in avoiding collisions with other objects in the environment.



**Alignment:** steer towards the average heading and speed of local flockmates

- Enforces cohesion to keep the flock together. Helps with collision avoidance, too.



**Cohesion:** steer to move toward the average position of local flockmates

- Agents at edge of the herd more vulnerable to predators
- Helps to keep the flock together

# Swarm Intelligence

- ❖ **Swarm intelligence (SI)** is an artificial intelligence technique based around the study of collective behavior in decentralized, self-organized systems.
- ❖ The express 'Swarm Intelligence' was originally used by Beni, Hackwood and Wang in 1989, in the context of cellular robotic systems to describe the self-organization of simple mechanical agents.
- ❖ It was later extended to include "any attempt to design algorithms or distributed problem-solving devices inspired by the collective behavior of social insect colonies and other animal societies"  
[Bonabeau, Dorigo, and Theraulaz, 1999]



# Swarm Intelligence (Cont'd)

- ❖ SI systems are typically made up of a population of simple agents interacting locally with one another and with their environment.
- ❖ Although there is normally no centralized control structure dictating how individual agents should behave, local interactions between such agents often lead to the emergence of global behavior.
- ❖ Sometimes called 'Collective Intelligence'.





# Components of SI

## ❖ **Agents:**

- Interact with the world and with each other (either directly or indirectly)

## ❖ **Simple behaviours**

- e.g. ants, termites, bees, wasps

## ❖ **Communication:**

- How agents interact with each other
- e.g. pheromones of ants

Simple behaviours of individual agents

+ Communication between a group of agents

= Emergent complex behaviour of the group of agents



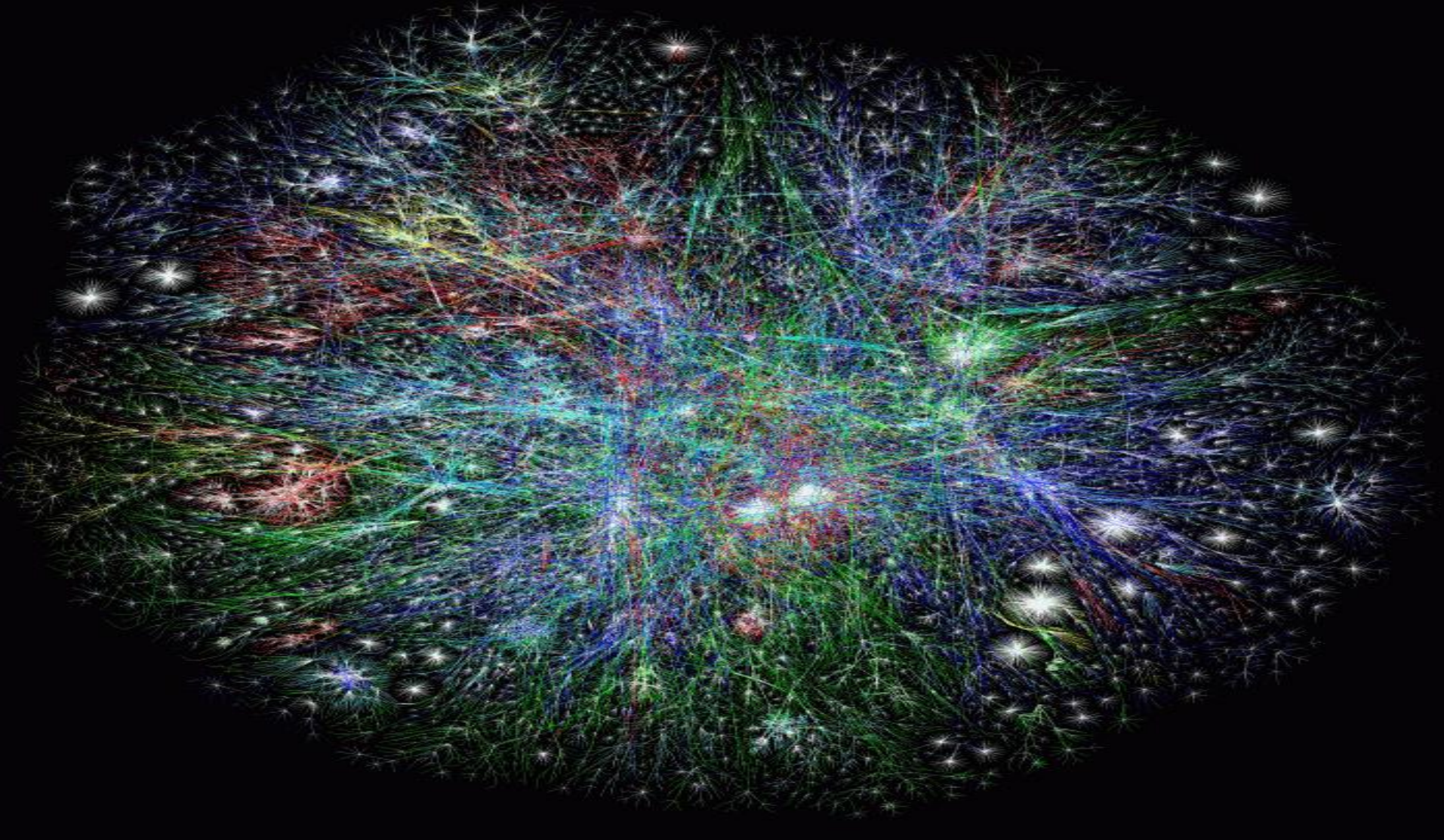
# Characteristics of SI

- ❖ **Distributed, no central control or data source**
- ❖ **Limited communication**
- ❖ **No (explicit) model of the environment**
- ❖ **Perception of environment (sensing)**
- ❖ **Ability to react to environment changes.**

Is SI relevant to people?



# The Web becomes a Giant Brain





# Part II: Communication





# The nature of communication

## Human communication

- Communication is the intentional exchange of information brought about by the production and perception of signs drawn from a shared system of conventional signs (AIMA, Russell&Norvig) → *language*
- Communication seen as an action (communicative act) and as an intentional stance

## Component steps of communication

Speaker

- ⇨ Intention
- ⇨ Generation
- ⇨ Synthesis

Hearer

- ⇨ Perception
- ⇨ Analysis
- ⇨ Disambiguation
- ⇨ Incorporation

→ { Syntax  
Semantics  
Pragmatics

# Artificial Communication

- **Computer communication**  
shared memory, message passing
- **Communication in SI** = more than simple communication, implies interaction  
The environment provides a computational infrastructure where interactions among agents take place. The infrastructure includes protocols for agents to communicate and protocols for agents to interact
- **Low-level Communication**: simple signals, traces
- **High-level Communication** : cognitive agents, mostly seen as intentional systems
- **Direct communication vs. Indirect communication**



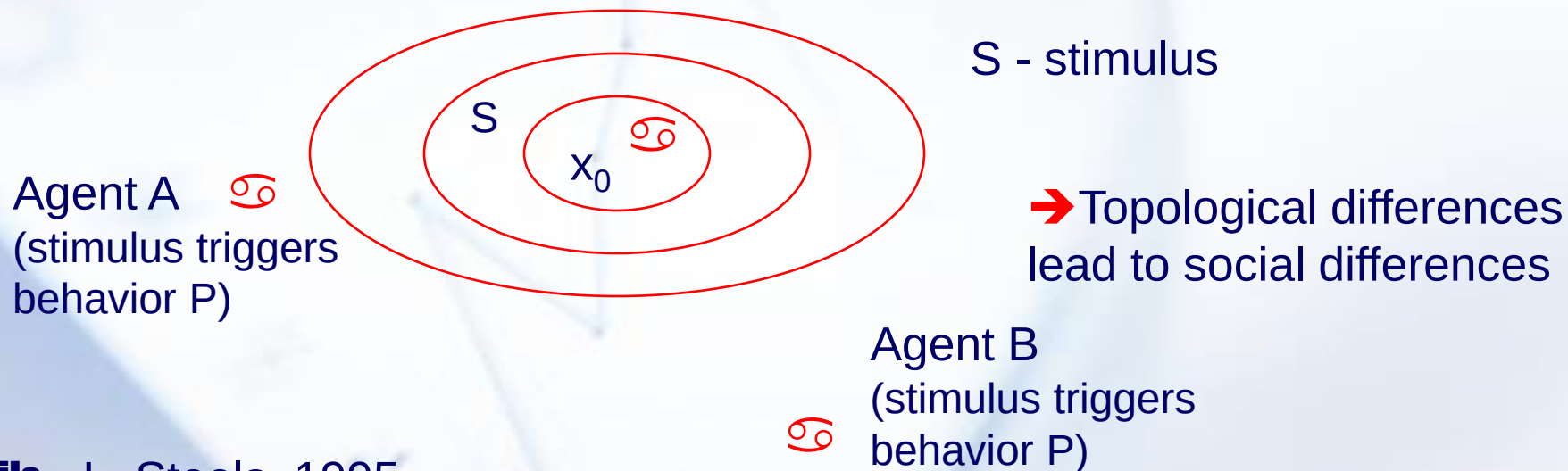
# (1) Indirect communication

## Signal propagation - Manta, A. Drogoul 1993

- An agent sends a signal, which is broadcast into the environment, and whose intensity decreases as the distance decreases
- At a point  $x$ , the signal may have one of the following intensities

$$V(x) = V(x_0) / \text{dist}(x, x_0)$$

$$V(x) = V(x_0) / \text{dist}(x, x_0)^2$$



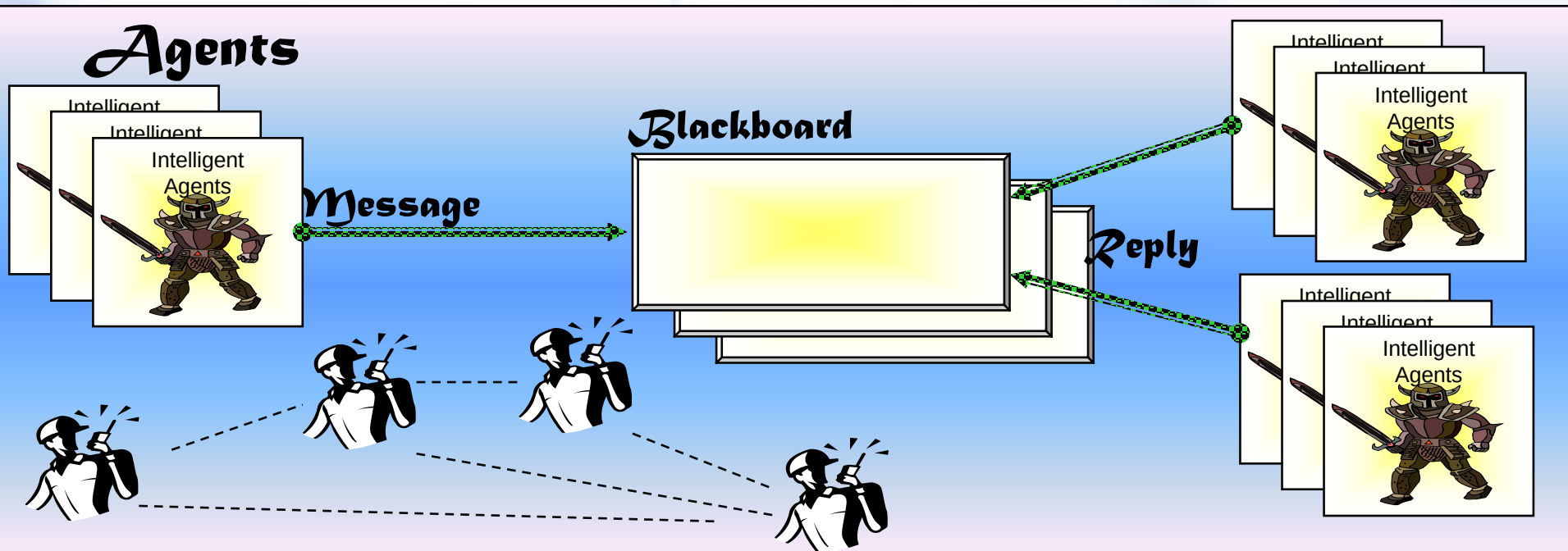
## Trails - L. Steels, 1995

- agents drop "radioactive crumbs" making trails
- an agent following a trail makes the trail faint until it disappears

# Indirect communication (cont'd)

## ◉ Blackboard systems

- a common area (shared memory) in which agents can exchange information, data, knowledge.
- Blackboard = a powerful distributed knowledge computation paradigm
- Agents = Knowledge sources (KS)







## (2) Direct communication

---

**Sending messages = method invocation**

- **The Actor language**

- an Actor executes a sequence of actions in reply to the received message

- **Exchange of partial plans**

- distributed planning

**ACL = Agent Communication Languages**

**communication as action - communicative acts**



# Theory of Speech Acts

*J. Austin - How to do things with words, 1962, J. Searle - Speech acts, 1969*

A speech act has 3 aspects:

- **Locution** = physical utterance by the speaker
- **Illocution** = the intended meaning of the utterance by the speaker (performative)
- **Prelocution** = the action that results from the locution
- Eg.,

Alice told Tom: "Would you please close the door"

locution

illocution

content

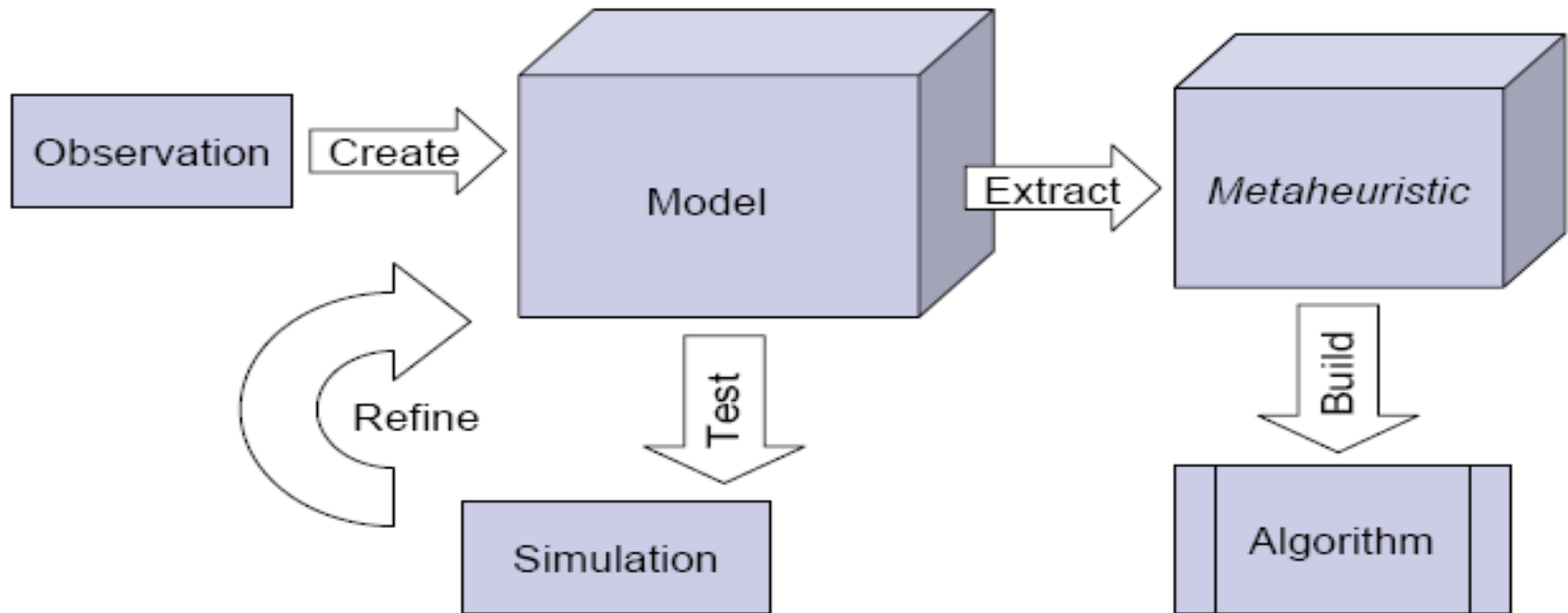
prelocution: door closed (hopefully!)

## ❖ Theoretical models: **Speech act theory**

## ❖ Practical models:

- shared languages  
like KIF, KQML, DAML
- service models  
like DAML-S
- social convention  
protocols

# Part III-IV : How to simulate SI for search?



**Example1: Ant --> Ant Colony Optimization (ACO)**

**Example2: Bird Flocking --> Particle Swarm Optimization (PSO)**







# **Part III: Ant Colony Optimization (ACO)**

First proposed by M. Dorigo, 1992



# Natural Ants

- ❖ **Individual ants are simple insects with limited memory and capable of performing simple actions.**
- ❖ **However, an ant colony expresses a complex collective behavior providing intelligent solutions to problems such as:**
  - carrying large items
  - forming bridges
  - **finding the shortest routes from the nest to a food source, prioritizing food sources based on their distance and ease of access.**

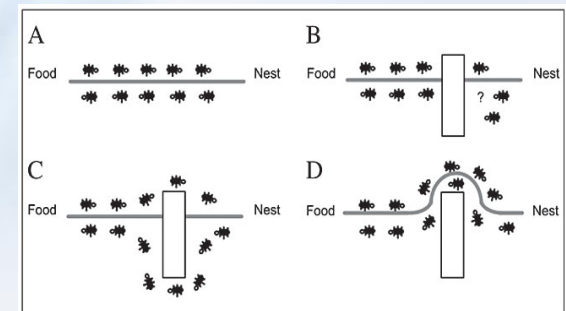


Figure 2. A, Ants in a pheromone trail between nest and food; B, an obstacle interrupts the trail; C, ants find two paths to go around the obstacle; D, a new pheromone trail is formed along the shorter path.

# Natural Ants

❖ **Moreover, in a colony each ant has its prescribed task, but the ants can switch tasks if the collective needs it.**

- Outside the nest, ants can have 4 different tasks:
  - **Foraging**: searching for and retrieving food
  - **Patrolling**: looking for food supply
  - **Midden work**: Sorting the colony refuse pile
  - **Nest maintenance work**: construction and clearing of chambers
- An ant's decision whether to perform a task depends on:
  - **The Physical State of the environment**:
    - If part of the nest is damaged, more ants do nest maintenance work to repair it
  - **Social Interactions with other ants**



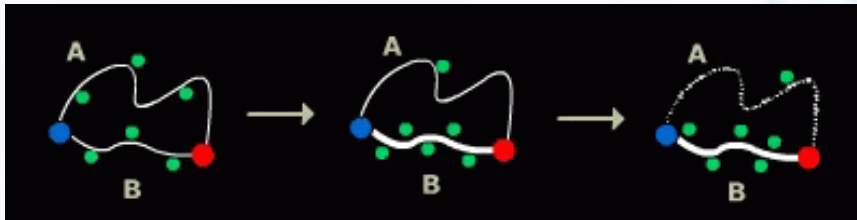
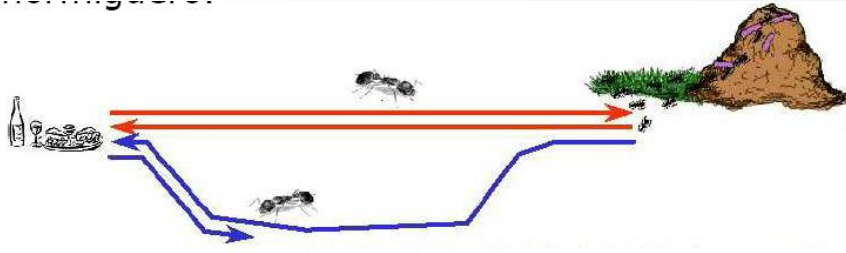
# How can the natural ants find the shortest path?

## ❖ They establish indirect communication system based on the deposition of pheromone over the path they follow.

- An isolated ant **moves at random**, but when it finds a pheromone trail, there is a high **probability** that this ant will decide to follow the trail.
- An ant foraging for food deposits pheromone over its route. When it finds a food source, it **returns to the nest reinforcing its trail**.
- So, other ants have greater probability to start following this trail and thereby laying more pheromone on it.
- This process works as a **positive feedback** loop system because the higher the intensity of the pheromone over a trail, the higher the probability of an ant start traveling through it.



# How can the natural ants find the shortest path?



- ❖ Since the route B is shorter, the ants on this path will complete the travel more times and thereby lay more pheromone over it.

- ❖ The pheromone concentration on trail B will increase at a higher rate than on A, and soon the ants on route A will choose to follow route B
- ❖ Since most ants will no longer travel on route A, and since the pheromone is volatile, trail A will start evaporating
- ❖ **Only the shortest route will remain!**





# Model Ant Colony for Optimization

- Each artificial ant is a probabilistic mechanism that constructs a solution to the problem, using:
  - Artificial pheromone deposition
  - Heuristic information: pheromone trails, and others
- Differences between real and artificial ants:
  - The pheromone is updated only after a solution has been constructed.
  - Additional mechanisms

---

**Algorithm 1** Ant colony optimization metaheuristic

---

Set parameters, initialize pheromone trails

**while** termination conditions not met **do**

    ConstructAntSolutions

    ApplyLocalSearch      {optional}

    UpdatePheromones

**end while**

---

# Model Ant Colony for Optimization (Cont'd)

## ❖ ConstructAntSolutions

- The rules for the probabilistic choice of solution components.

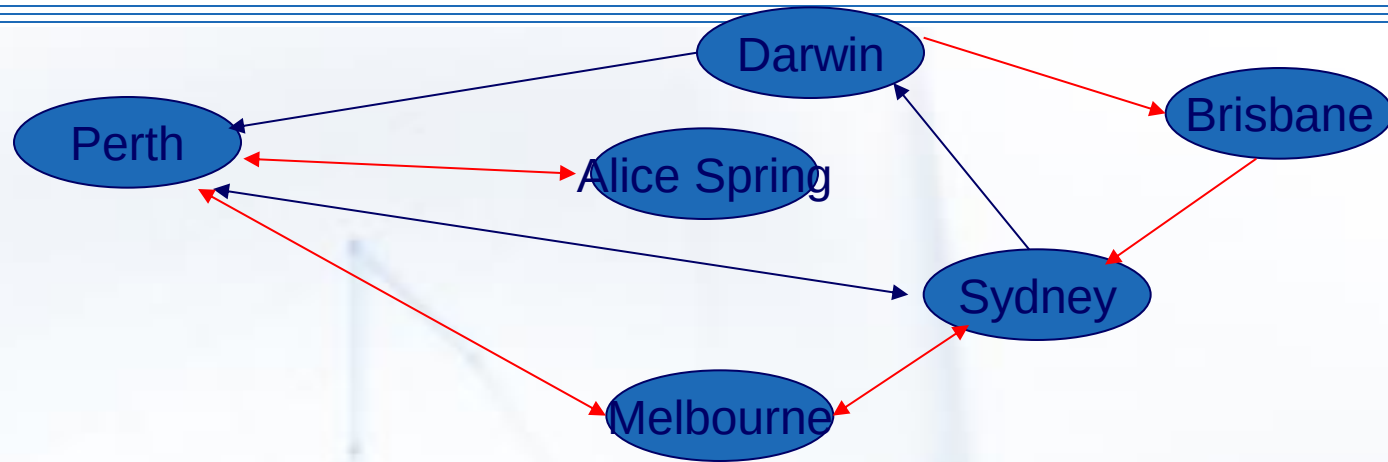
## ❖ UpdatePheromones

- It is used to increase the pheromone values associated with good or promising solutions, and decrease those that are associated with bad ones.
  - Decreasing all the pheromone values through pheromone evaporation --> allows “forgetting”-> favors exploration of new areas
  - Increasing the pheromone levels associated with a chosen set of good solutions -> makes the algorithm converge to a solution

The exact methods vary across different ACO variants



# Ant System (AS): The first ACO Algorithm (TSP as example)



## ❖ ConstructAntSolutions

$$p_{ij}^k(t) = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha [\eta_{ij}(t)]^\beta}{\sum_{l \in N_i^k(t)} [\tau_{il}(t)]^\alpha [\eta_{il}(t)]^\beta}, & j \in N_i^k(t) \\ 0, & j \notin N_i^k(t) \end{cases}$$

Quantity of  
pheromone

Heuristic  
distance

$\alpha, \beta$  constants

## ❖ UpdatePheromones

Evaporation rate

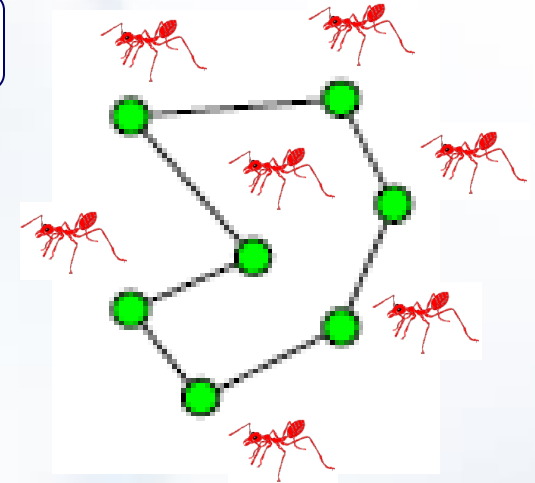
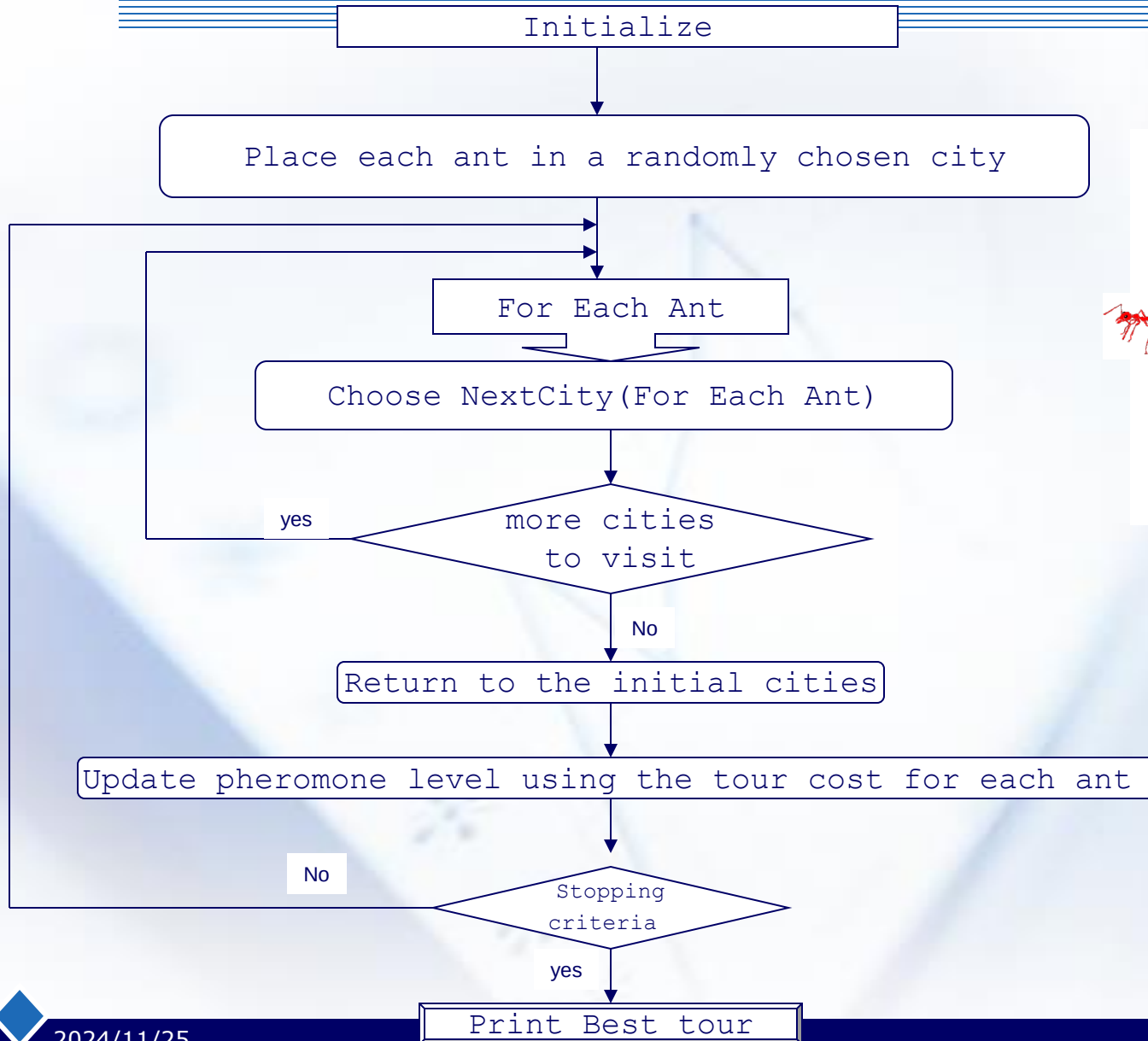
$$\tau_{ij}(t+1) = (1 - \rho) \cdot \tau_{ij}(t) + \Delta \tau_{ij}(t)$$

$$\Delta \tau_{ij}(t) = \sum_{k=1}^m \Delta \tau_{ij}^k(t)$$

Pheromone laid by  
each ant that uses  
edge (i,j)

$$\Delta \tau_{ij}^k(t) = \begin{cases} Q/L^k, & \text{第} k \text{只蚂蚁在本次遍历中经过路径 } c_i - c_j \\ 0, & \text{其他} \end{cases}$$

# AS for Travel Salesman Problem (TSP)



Flow chart after  
B.Ombuki-  
Berman: swarm  
intelligence



# Trivial Example of TSP (4 Cities)

Two ants

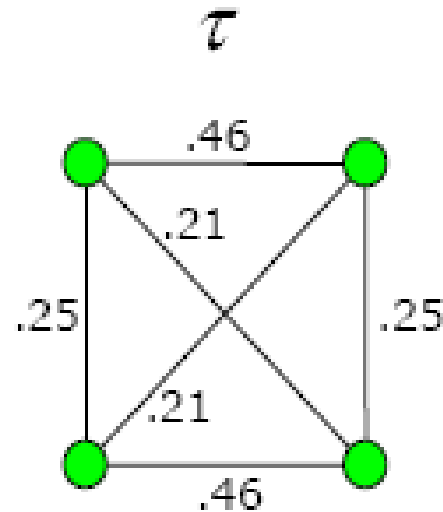
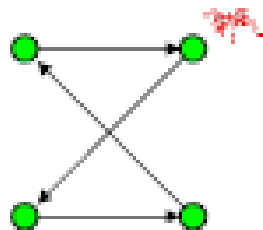
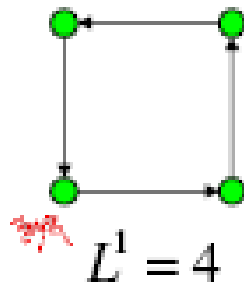


Image from Olle Gallmo:  
Swarm Intelligence

# Another Example: Job Shop Scheduling (JSSP)

- ❖ **Difficult NP-Hard Optimization Problem**
- ❖ **Involves assigning jobs to machines**
- ❖ **Jobs broken into tasks**
  - Same number of tasks per job as machines
  - Each task has processing time
- ❖ **Task constraints and limitations**
  - Processed according to predefined order
  - No concurrent processing
  - No-preemption

# Job Shop Scheduling

## Example Instance: (machine, duration)

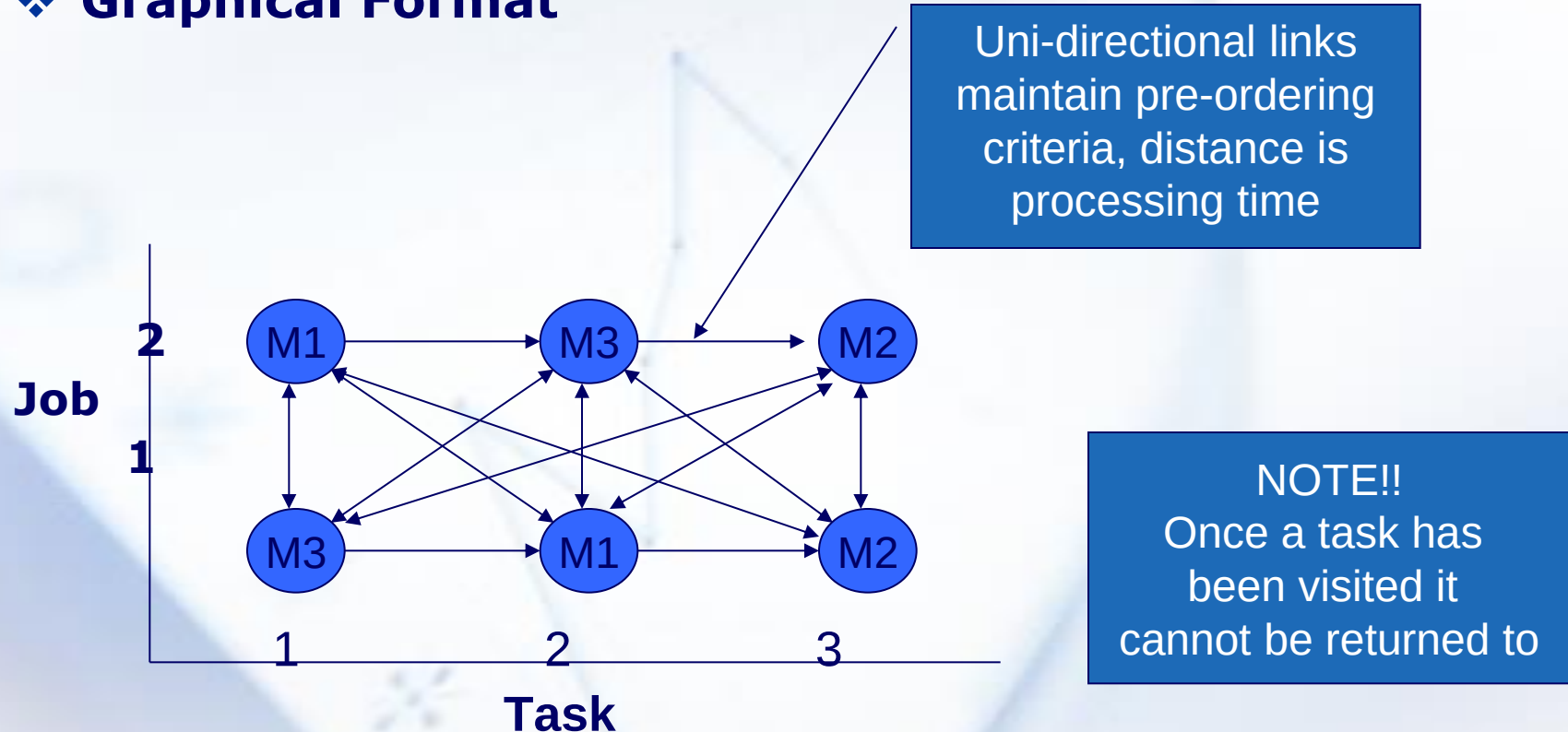
|       |        |       |       |
|-------|--------|-------|-------|
| Job 1 | (2,10) | (1,2) | (3,5) |
| Job 2 | (1,12) | (3,6) | (2,2) |

## Possible Schedule (not optimal)

|                |       |       |       |       |    |
|----------------|-------|-------|-------|-------|----|
| M <sub>1</sub> |       | Job 1 | Job 2 |       | 24 |
| M <sub>2</sub> | Job 1 |       |       | Job 2 | 32 |
| M <sub>3</sub> |       |       | Job 2 | Job 1 | 35 |

# Ant System for JSSP

## ❖ Graphical Format



## ❖ Ant System tends to converge quickly

- This means that its **exploitation** of the best solution found is too high, it should be **exploring** solution space more
- Pheromone evaporation/update rule (better rule may exist)

## ❖ Led to extensions of the ant system

- *Elitist Strategy for Ant Systems (EAS)*
- *Rank based Ant Systems ( $AK_{RANK}$ )*
- *MAX-MIN Ant system (MMAS)*
- *Ant Colony System (ACS)*



- the elite or best ant has its trail updated with additional pheromone. The net effect is to bias trails of superior quality.

## AS Elitist Cycle

1. Distribute ant agents on different cities
2.  $i = 0$
3. While  $k < \# \text{ ants}$   
     $ant_k$  adds new city based on  $P_{ij}$   
End While
4. If  $i++ < \# \text{ cities}$  GOTO 3
5. All ants compute  $\Delta \tau_{ij}$
6.  $\tau_{ij} = \rho * \tau_{ij} + \Delta \tau_{ij}^*(globalbest)$



- ants are ranked according to the goodness of their solution

## AK<sub>RANK</sub>

1. Distribute ant agents on different cities
2.  $i = 0$
3. While  $k < \# \text{ ants}$   
 $\text{ant}_k$  adds new city based on  $P_{ij}$   
 End While
4. If  $i++ < \# \text{ cities}$  GOTO 3
5. Sort ants ascending by cost (Ant\_list)
6. For  $i = 0; i < RK$

$$\Delta \tau_{ij} + (RK - i) \Delta \tau_{ij} i$$

7.  $\tau_{ij} = \rho * \tau_{ij} + \Delta \tau_{ij}^* (\text{globalbest})$

- An improvement over the original Ant System to allow for more exploration
  - Introduced use of tending to global best from iteration best solution over time
    - i.e., **only the best ant updates the pheromone trails, and that,**
    - **the value of the pheromone is bound, upper and lower bound imposed**
      - bounds on pheromone that are dependant on solution quality
  - Bounds set empirically

## Ant Min Max system (ACSM) Cycle

1. Distribute ant agents on different cities

$$2. \tau_{\min} = \frac{C}{avg. * n^2}$$

3.  $i = 0$

4. While  $k < \# \text{ ants}$

$ant_k$  adds new city based on  $P_{ij}$

End While

5. If  $i++ < \# \text{ cities}$  GOTO 3

6. Improve  $N$  solutions

$$7. \tau_{ij} = \rho * \tau_{ij} + \Delta \tau_{ij}^{* (globalbest)}$$

8. Apply min max rules  $\tau_{\min} \leq \tau_{ij} \leq \tau_{\max}$

9. If solution improved update  $\tau_{\max} = \frac{1}{bestsolution * (1 - \rho)}$

- Introduction of a **local pheromone update** in addition to the pheromone update performed at the end of the construction process (known as **offline pheromone update**)

### Ant Colony System

1. Distribute ant agents on different cities
2.  $i = 0$
3. While  $k < \# \text{ ants}$ 
  - $ant_k$  adds new city based on  $P_{ij}$  and  $\alpha q_0$
  - $ant_k$  updates the trail locallyEnd While
4. If  $i++ < \# \text{ cities}$  GOTO 3
5.  $\tau_{ij} = \rho * \tau_{ij} + \Delta \tau_{ij}^*(globalbest)$



# **Part III : Particle Swarm Optimization (PSO)**

Firstly Proposed by Kennedy and Eberhart , 1995

“Once again, nature has provided us with a technique for processing information that is at once elegant and versatile”



# Bird flocking

- ❖ In PSO, a “swarm” is defined as an apparently disorganized collection (**population**) of moving individuals that tend to cluster together while each individual seems to be moving in a random direction.



Bird flocking is one of the best example of PSO in nature.

# Modeling bird flocking

❖ **The synchrony of flocking behavior is thought to be a function of bird's efforts to maintain an optimal distance between themselves and their neighbors.**

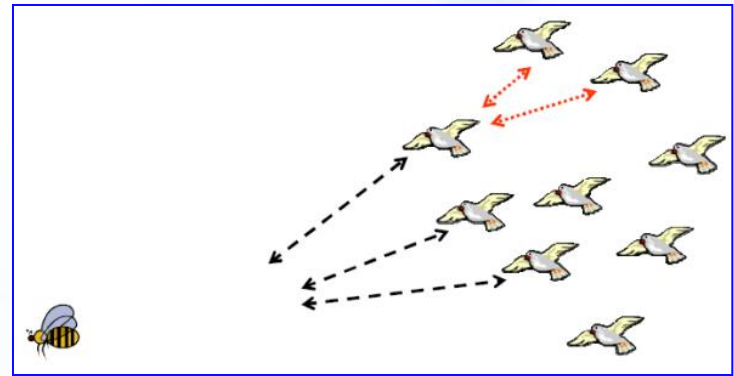
- Birds and fish adjust their physical movement to avoid predators, seek for food and mates.
- Humans tend to adjust our beliefs and attitudes to conform with those of our social peers. Humans change in abstract multidimensional space, collision-free.





# From Bird to Particle

- ❖ Imagine a bird's flock in an area where there is a single food source.
- ❖ A bird don't know where the food is, but it knows its distance to the food.
- ❖ The best strategy is to follow the bird that is closer to the food.



- ❖ **Particles** save and communicate the best solution they have found.



# Features of PSO

- ❖ **Population initialized by assigning random positions and velocities; potential solutions are then *flown* through hyperspace.**
- ❖ **Each particle keeps track of its “best” (highest fitness) position in hyperspace.**
  - This is called “pBest” for an individual particle
  - It is called “gBest” for the best in the population
  - It is called “lBest” for the best in a defined neighborhood
- ❖ **At each time step, each particle stochastically accelerates toward its pBest and gBest (or lBest).**





# Particle Swarm Optimization Process

---

**Step1. Initialize population in hyperspace.**

**Step2. Evaluate fitness of individual particles.**

**Step3. Modify velocities based on previous best and global (or neighborhood) best.**

**Step4. Terminate on some condition.**

**Step5. Go to step 2.**



# How do particles fly?

## ❖ Combination of gBest and the pBest (lBest)

- need a compromise

## ❖ lBest can be:

- **Social**: the particles around are always the same, no matter where they are in space
- **Geographical**: the particles around are those whose distance is the shortest

## ❖ Global PSO vs. Local PSO

- the global version converges quickly to a solution but it gets more easily stuck in local minima.



# PSO Velocity Update Equations

## ❖ Global Version:

$$v_{ik}^{t+1} = \omega v_{ik}^t + c_1 \xi (p_{ik}^t - x_{ik}^t) + c_2 \eta (p_{gk}^t - x_{ik}^t), k = 1, 2, \dots, d$$

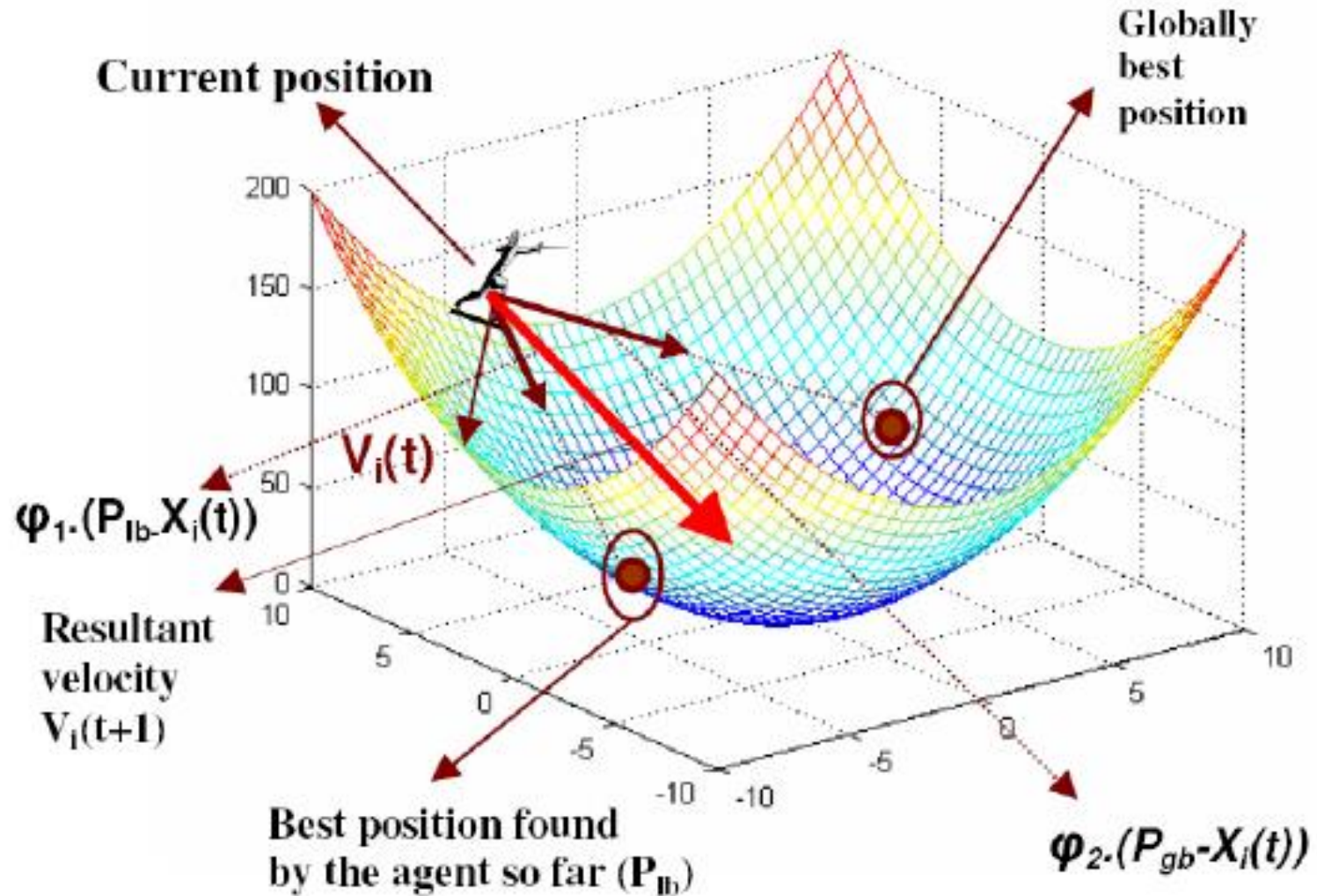
$$x_{ik}^{t+1} = x_{ik}^t + v_{ik}^{t+1}, k = 1, 2, \dots, d$$

where  $k$  is the dimension,  $c_1$  and  $c_2$  are positive constants,  $\xi$  and  $\eta$  are random functions, and  $\omega$  is the inertia weight.

For neighborhood version, change  $p_{gk}$  to  $p_{lk}$ .



# Illustrating the velocity update schema of global PSO





# PSO: Related issues

- ❖ **Controlling velocities (determining the best value for  $V_{max}$ )**
  - Usually set maximum velocity to dynamic range of variable
- ❖ **Usually set  $c_1$  and  $c_2$  to 2**
- ❖ **Inertia weight influences tradeoff between global and local exploration.**
  - Good approach is to reduce inertia weight during run (i.e., from 0.9 to 0.4 over 1000 generations)
- ❖ **Swarm Size and Neighborhood Size**



# Advantages of PSO

- ❖ **Adaptation operates on velocities**
  - Most other methods operate on positions
  - Effect: PSO has a builtin momentum
  - Particles tend to hurdle past optima – an advantage, since the best positions are remembered anyway
- ❖ **Simple in concept**
- ❖ **Easy to implement**
- ❖ **Computationally efficient**
- ❖ **Effective on a variety of problems**



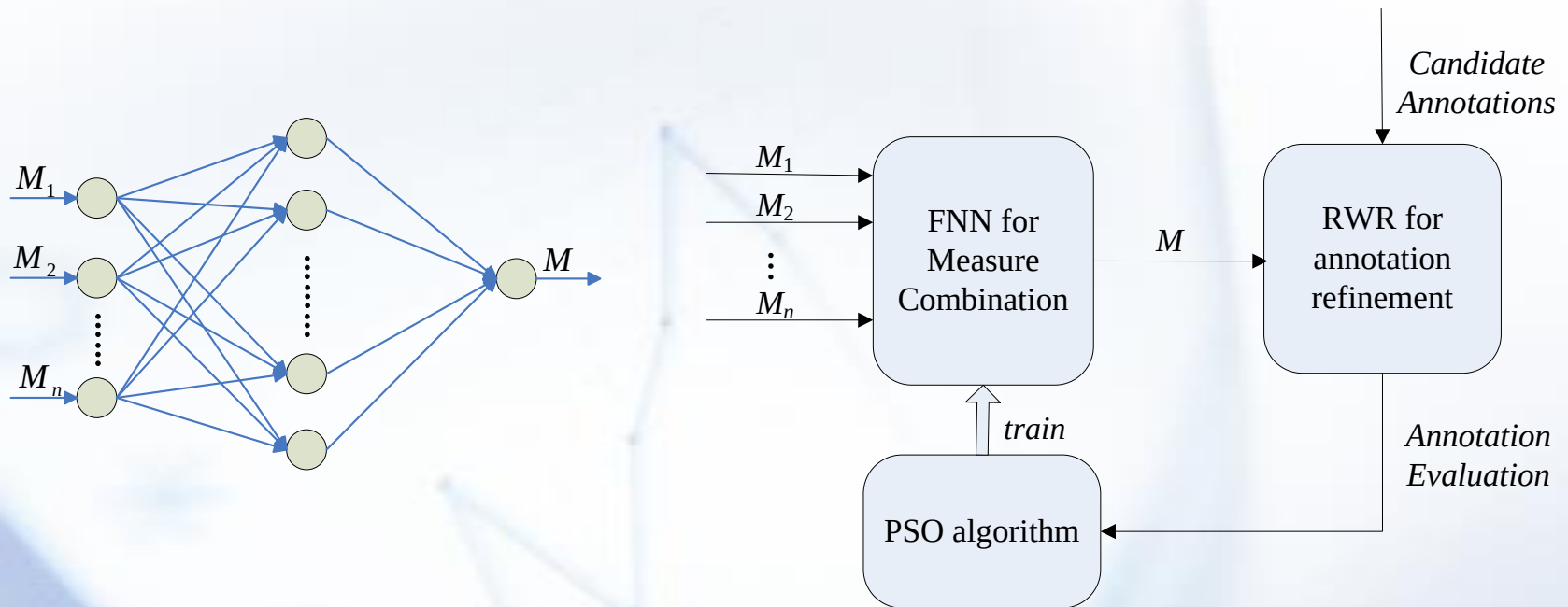
# An application of PSO – Image Annotation Refinement

|              |                                                                                   |                                                                                    |                                                                                     |                                                                                     |
|--------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Images       |  |  |  |  |
| Ground-truth | bear, polar, snow, tundra                                                         | bear, face, polar, snow                                                            | aerial, bear, polar, snow                                                           | bear, ice, polar, snow                                                              |
| MMP          | snow, tundra, cow, bear, plane                                                    | snow, locomotive, train, bear, pillar                                              | snow, plane, bear, pillar, hills                                                    | snow, plane, hills, bear, sand                                                      |
| MMP+JNC      | rocks, stone, cow, snow, grass                                                    | stone, rocks, snow, grass, locomotive                                              | stone, rocks, grass, snow, mountain                                                 | stone, rocks, grass, smoke, snow                                                    |
| MMP+LIN      | cow, snow, bear, plane, flowers                                                   | snow, locomotive, train, bear, pillar                                              | snow, plane, bear, pillar, wall                                                     | snow, plane, hills, stone, bear                                                     |
| MMP+BNP      | snow, tundra, cow, bear, plane                                                    | snow, locomotive, train, bear, pillar                                              | snow, plane, bear, pillar, hills                                                    | snow, plane, hills, bear, sand                                                      |
| MMP+hybrid   | cow, snow, tundra, polar, bear                                                    | snow, locomotive, polar, train, bear                                               | snow, polar, plane, bear, pillar                                                    | snow, plane, polar, bear, hills                                                     |

**Table 2.** Image annotation accuracy by using various semantic similarity measures, where the 'NumKeyword' means the number of keywords with non-zero recall, 'IR\_F1' means the increase rate of F1 measure.

| Annotation Method | NumKeyword | Recall | Precision | F1     | IR_F1  |
|-------------------|------------|--------|-----------|--------|--------|
| MMP               | 121        | 0.1665 | 0.2706    | 0.2061 | 7.81%  |
| MMP+JNC           | 77         | 0.1025 | 0.1838    | 0.1316 | 68.85% |
| MMP+LIN           | 116        | 0.1673 | 0.2689    | 0.2063 | 7.71%  |
| MMP+BNP           | 122        | 0.1725 | 0.2739    | 0.2117 | 4.96%  |
| MMP+hybrid        | 124        | 0.1817 | 0.2860    | 0.2222 | —      |

# An application of PSO – Image Annotation Refinement



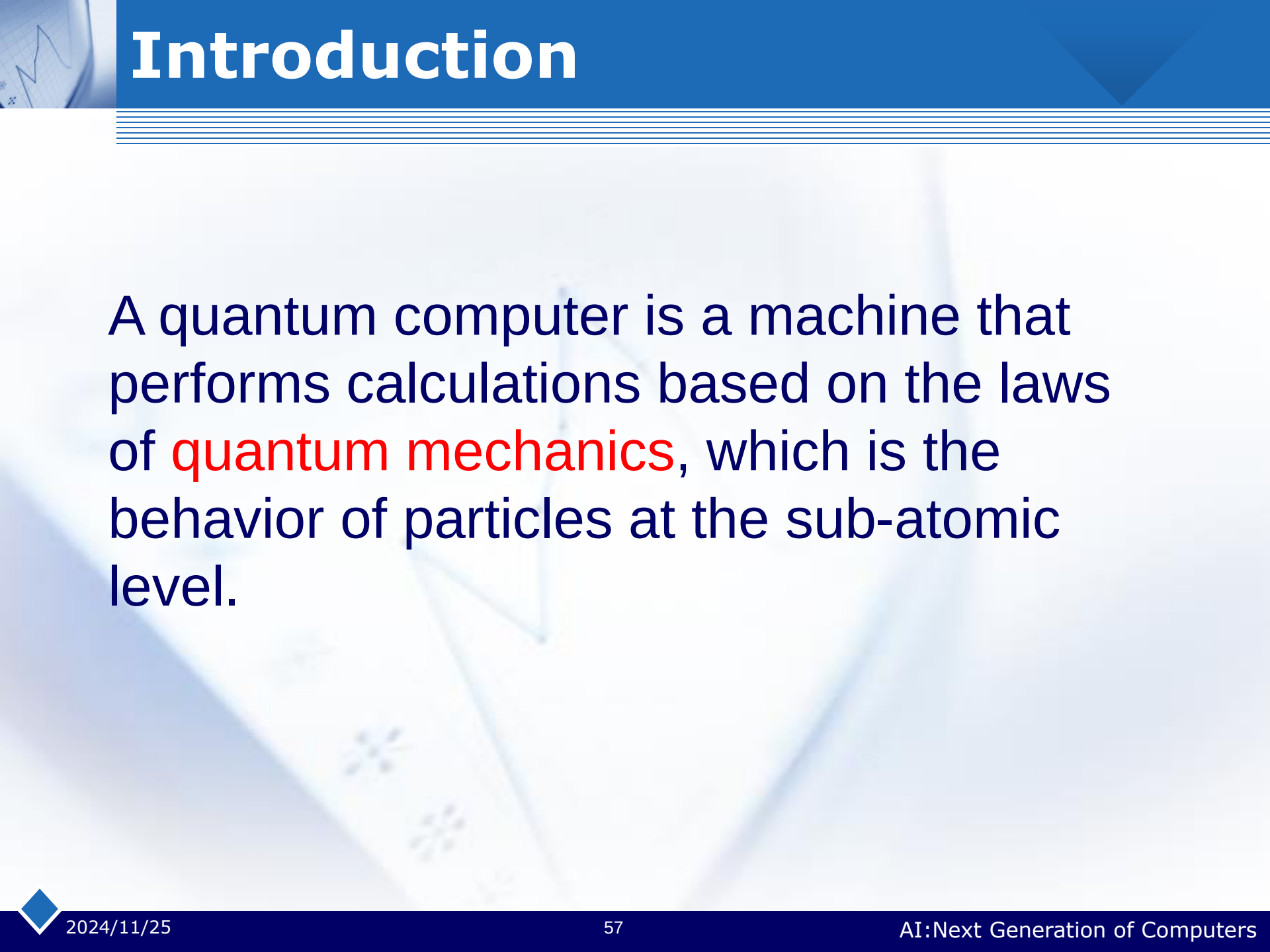
PSO is used to optimize the feedback neural network for combining the similarity measures between keywords.

Y. Cao et al., Using Neural Network to Combine Measures of Word Semantic Similarity for Image Annotation, ICIA 2011



# **Supplement: Quantum Computing**





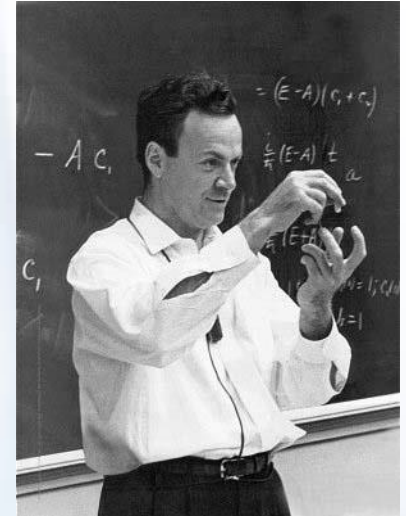
# Introduction

A quantum computer is a machine that performs calculations based on the laws of **quantum mechanics**, which is the behavior of particles at the sub-atomic level.



# Brief History

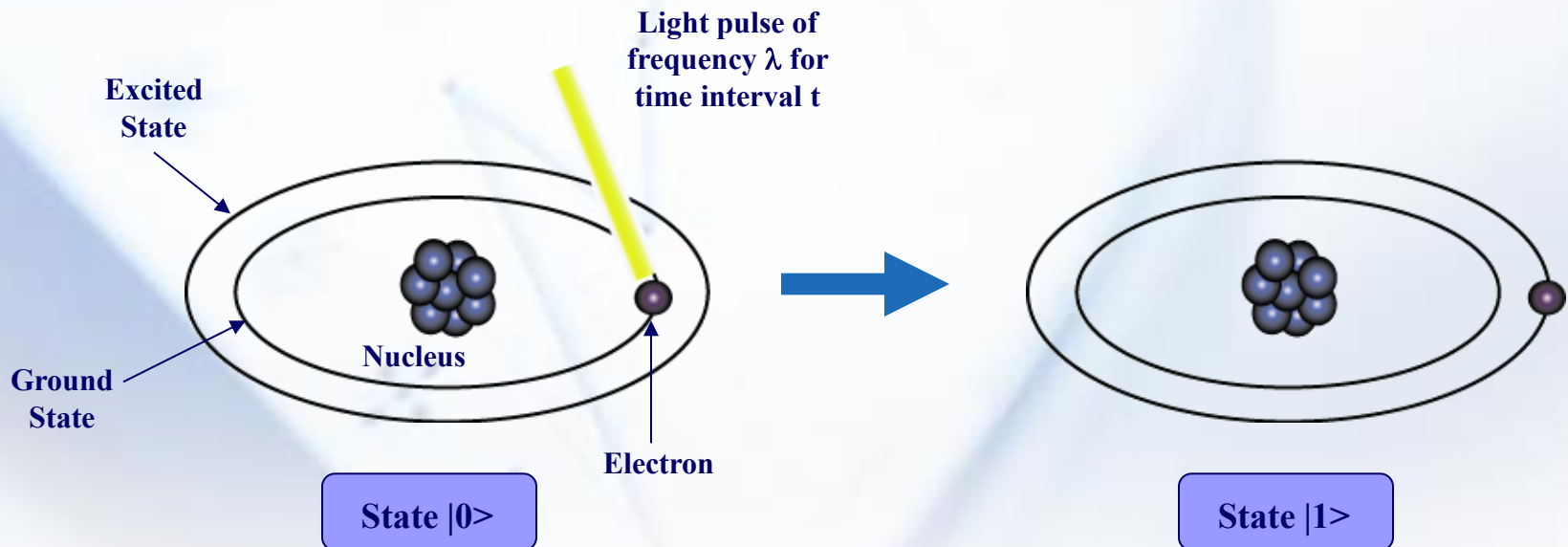
- “I think I can safely say that nobody understands quantum mechanics” - Feynman
- 1982 - Feynman proposed the idea of creating machines based on the laws of quantum mechanics instead of the laws of classical physics.
- 1985 - David Deutsch developed the quantum turing machine, showing that quantum circuits are universal.
- 1994 - Peter Shor came up with a quantum algorithm to factor very large numbers in polynomial time.





# Representation of Data - Qubits

- A bit of data is represented by a single atom that is in one of two states denoted by  $|0\rangle$  and  $|1\rangle$ . A single bit of this form is known as a *qubit*
- A physical implementation of a qubit could use the two energy levels of an atom. An excited state representing  $|1\rangle$  and a ground state representing  $|0\rangle$ .



# Representation of Data - Superposition

A single qubit can be forced into a *superposition* of the two states denoted by the addition of the state vectors:

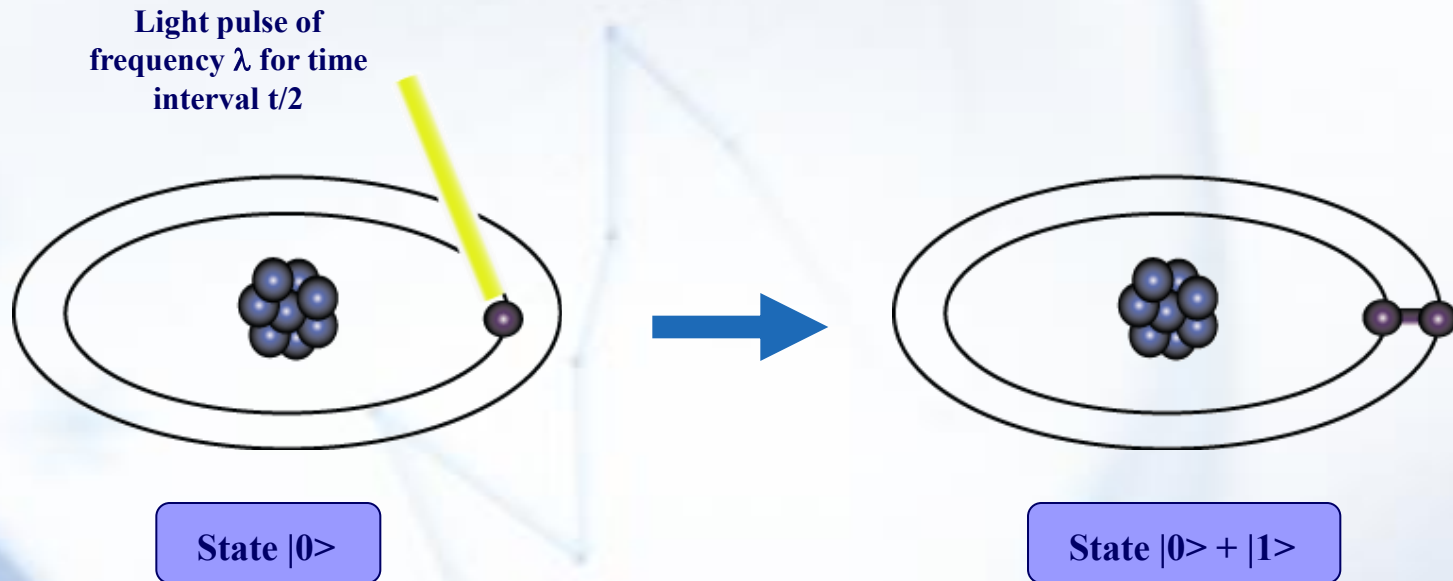
$$|\psi\rangle = \alpha_1 |0\rangle + \alpha_2 |1\rangle$$

Where  $\alpha_1$  and  $\alpha_2$  are complex numbers and  $|\alpha_1|^2 + |\alpha_2|^2 = 1$

A qubit in superposition is in both of the states  $|1\rangle$  and  $|0\rangle$  at the same time



# Representation of Data - Superposition



# Data Retrieval

- In general, an  $n$  qubit register can represent the numbers 0 through  $2^n - 1$  simultaneously.

**Sound too good to be true?...It is!**

- If we attempt to retrieve the values represented within a superposition, the **superposition randomly collapses** to represent just one of the original values.

In our equation:  $|\psi\rangle = \alpha_1|0\rangle + \alpha_2|1\rangle$ ,  $\alpha_1$  represents the probability of the superposition collapsing to  $|0\rangle$ . The  $\alpha$ 's are called probability amplitudes.



# Relationships among data - Entanglement

- ***Entanglement*** is the ability of quantum systems to exhibit correlations between states within a superposition.
- Imagine two qubits, each in the state  $|0\rangle + |1\rangle$  (a superposition of the 0 and 1.) We can entangle the two qubits such that the measurement of one qubit is always correlated to the measurement of the other qubit.



# Shor's Algorithm

- Shor's algorithm shows (in principle,) that a quantum computer is capable of factoring very large numbers in polynomial time.

The algorithm is dependant on

- Modular Arithmetic
- Quantum Parallelism
- Quantum Fourier Transform



# Shor's Algorithm - Periodicity

- An important result from Number Theory:

$F(a) = x^a \bmod N$  is a periodic function

- Choose  $N = 15$  and  $x = 7$  and we get the following:

$$7^0 \bmod 15 = 1$$

$$7^1 \bmod 15 = 7$$

$$7^2 \bmod 15 = 4$$

$$7^3 \bmod 15 = 13$$

$$7^4 \bmod 15 = 1$$

⋮





# Shor's Algorithm - In Depth Analysis

**To Factor an odd integer  $N$  (Let's choose 15) :**

1. Choose an integer  $q$  such that  $N < q < 2^N$  **let's pick 256**
2. Choose a random integer  $x$  such that  $\text{GCD}(x, N) = 1$  **let's pick 7**
3. Create two quantum registers (these registers must also be entangled so that the collapse of the input register corresponds to the collapse of the output register)
  - Input register: must contain enough qubits to represent numbers as large as  $q-1$ . **up to 255, so we need 8 qubits**
  - Output register: must contain enough qubits to represent numbers as large as  $N-1$ . **up to 14, so we need 4 qubits**



# Shor's Algorithm - Preparing Data

4. Load the input register with an equally weighted superposition of all integers from 0 to  $q-1$ . **0 to 255**
5. Load the output register with all zeros.

**The total state of the system at this point will be:**

$$\frac{1}{\sqrt{256}} \sum_{a=0}^{255} |a\rangle, |0000\rangle$$

Input  
Register

Output  
Register

Note: the comma here denotes that the registers are entangled

# Shor's Algorithm - Modular Arithmetic

6. Apply the transformation  $x^a \bmod N$  to each number in the input register, storing the result of each computation in the output register.

| Input Register | $7^a \bmod 15$ | Output Register |
|----------------|----------------|-----------------|
| $ 0\rangle$    | $7^0 \bmod 15$ | 1               |
| $ 1\rangle$    | $7^1 \bmod 15$ | 7               |
| $ 2\rangle$    | $7^2 \bmod 15$ | 4               |
| $ 3\rangle$    | $7^3 \bmod 15$ | 13              |
| $ 4\rangle$    | $7^4 \bmod 15$ | 1               |
| $ 5\rangle$    | $7^5 \bmod 15$ | 7               |
| $ 6\rangle$    | $7^6 \bmod 15$ | 4               |
| $ 7\rangle$    | $7^7 \bmod 15$ | 13              |



# Shor's Algorithm - Superposition Collapse

7. Now take a measurement on the output register. This will collapse the superposition to represent *just one* of the results of the transformation, let's call this value  $c$ .

Our output register will collapse to represent one of the following:

$|1\rangle$ ,  $|4\rangle$ ,  $|7\rangle$ , or  $|13\rangle$

For sake of example, let's choose  $|1\rangle$



# Shor's Algorithm - Entanglement

*Now things really get interesting !*

8. Since the two registers are entangled, measuring the output register will have the effect of partially collapsing the input register into an **equal superposition** of each state between 0 and  $q-1$  that yielded  $c$  (the value of the collapsed output register.)

Since the output register collapsed to  $|1\rangle$ , the input register will partially collapse to:

$$\frac{1}{\sqrt{64}} |0\rangle + \frac{1}{\sqrt{64}} |4\rangle + \frac{1}{\sqrt{64}} |8\rangle + \frac{1}{\sqrt{64}} |12\rangle, \dots$$

The probabilities in this case are  $\frac{1}{\sqrt{64}}$  since our register is now in an equal superposition of 64 values (0, 4, 8, ... 252)



# Shor's Algorithm - QFT

9. We now apply the Quantum Fourier transform on the partially collapsed input register. The fourier transform has the effect of taking a state  $|a\rangle$  and transforming it into a state given by:

$$\frac{1}{\sqrt{q}} \sum_{c=0}^{q-1} e^{2\pi i ac/q} |c\rangle, q = 256$$



# Shor's Algorithm - QFT

$$\frac{1}{\sqrt{64}} \sum_{a \in A} |a\rangle, |1\rangle \longrightarrow \frac{1}{\sqrt{256}} \sum_{c=0}^{255} e^{2\pi i ac/256} |c\rangle$$

**Note:** A is the set of all values that  $7^a \bmod 15$  yielded 1.  
In our case  $A = \{0, 4, 8, \dots, 252\}$

So the final state of the input register after the QFT is:

$$\frac{1}{\sqrt{64}} \sum_{a \in A} \frac{1}{\sqrt{256}} \sum_{k=0}^{255} e^{2\pi i ak/256} |k\rangle, |1\rangle$$





# Shor's Algorithm - QFT

The QFT will essentially peak the probability amplitudes at integer multiples of  $q/4$  in our case  $256/4$ , or 64.

$$|0\rangle, |64\rangle, |128\rangle, |192\rangle$$

So we no longer have an equal superposition of states, the probability amplitudes of the above states are now higher than the other states in our register. We measure the register, and it will collapse with high probability to one of these multiples of 64, let's call this value  $m$ .

With our knowledge of  $q$ , and  $m$ , there are methods of calculating the period (one method is the continuous fraction expansion of the ratio between  $q$  and  $m$ .)



# Obtaining the order by continued fractions

- ❖ Suppose we obtain  $m=192$  from the measurement; computing the continued fraction expansion thus gives

$$192/256 = 1/(1+1/3)$$

- ❖ So that  $3/4$  occurs as convergent in the expansion, giving  $r = 4$  as the order of  $x = 7$ .
- ❖ Check to see whether  $r = 4$  is the order by calculating whether

$$x^r \bmod N = 1$$



# Shor's Algorithm - The Factors

10. Now that we have the period, the factors of  $N$  can be determined by taking the greatest common divisor of  $N$  with respect to  $x^{(P/2) + 1}$  and  $x^{(P/2) - 1}$ . The idea here is that this computation will be done on a classical computer.

We compute:

$$\text{Gcd}(7^{4/2} + 1, 15) = 5$$

$$\text{Gcd}(7^{4/2} - 1, 15) = 3$$

**We have successfully factored 15!**



# Shor's Algorithm - Problems

- The QFT comes up short and reveals the wrong period. This probability is actually dependant on your choice of  $q$ . The larger the  $q$ , the higher the probability of finding the correct probability.
- The period of the series ends up being odd

If either of these cases occur, we go back to the beginning and pick a new  $x$ .



# Quantum Computer

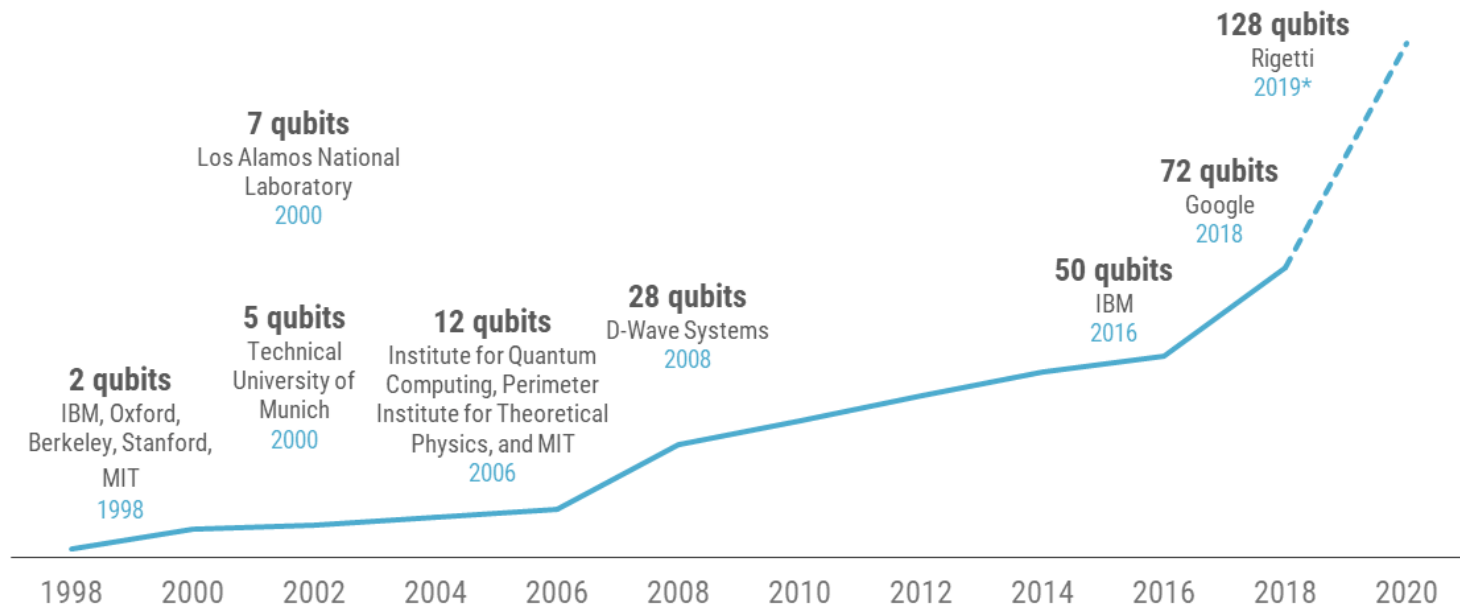
- ❖ 将微晶体管压缩到原子般大小，然后在极小的面积上放入数十亿颗量子微晶体管，进而利用量子态的叠加性和相干性进行信息运算、保存和处理
- ❖ 可能的实现方式：核磁共振，量子点，离子阱，半导体硅基，**Josephson**结，相移片.....

# Quantum Computer



## Quantum computers are getting more powerful

Number of qubits achieved by date and organization 1998 – 2020\*

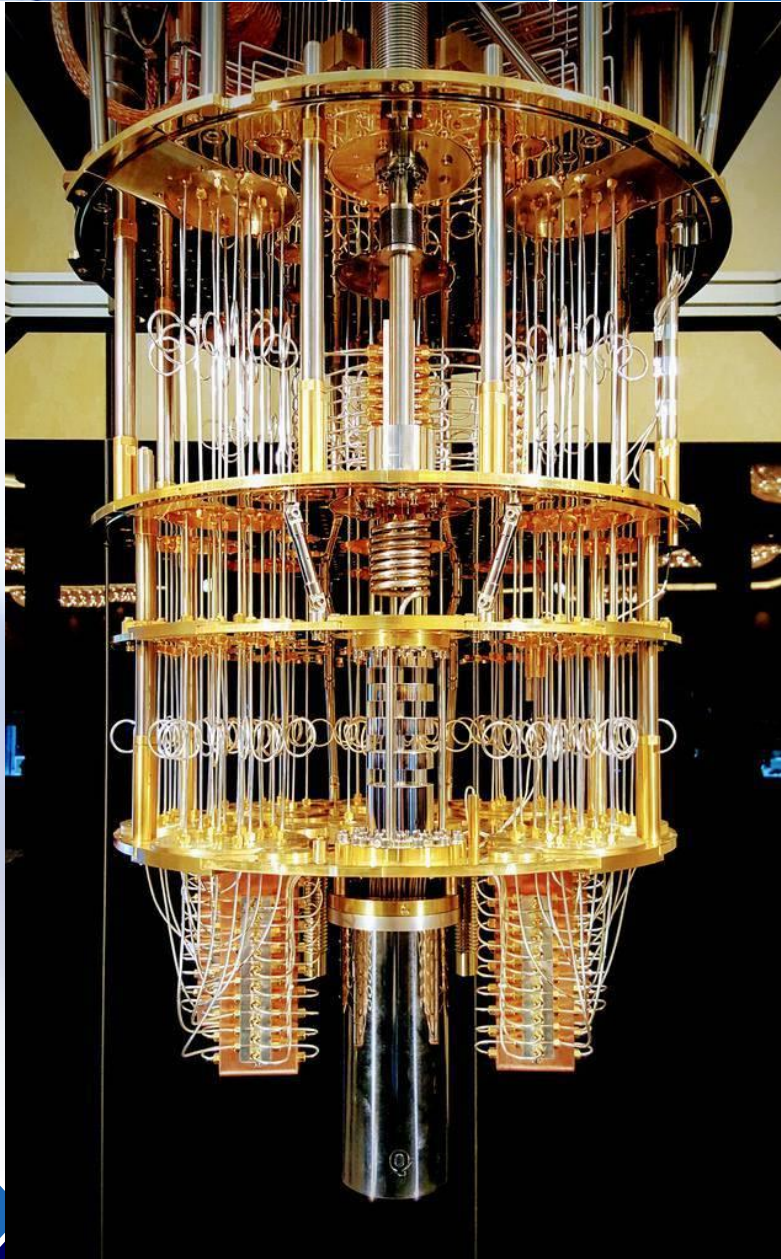


Source: MIT, Qubit Counter. \*Rigetti quantum computer expected by late 2019.

CBINSIGHTS



# We are close to a universal quantum computer, here's where we are at



One of two amplifying stages is cooled to a temperature of 4 Kelvin.

## Inside Look: Quantum Computer

Harnessing the power of a quantum processor requires maintaining constant temperatures near absolute zero. Here's a look at how a dilution refrigerator, made from more than 2,000 components, exploits the mixing properties of two helium isotopes to create such an environment.

In order to minimize energy loss, the coaxial lines that direct signals between the first and second amplifying stages are made out of superconductors.

Quantum amplifiers inside of a magnetic shield capture and amplify processor readout signals while minimizing noise.

QUBIT SIGNAL AMPLIFIER

INPUT MICROWAVE LINES

THE MARCH TO ABSOLUTE ZERO  
(or minus 459.67 degrees Fahrenheit)

4 KELVIN

Attenuation is applied at each stage in the refrigerator in order to protect qubits from thermal noise during the process of sending control and readout signals to the processor.

800 MILLIKELVINS

100 MILLIKELVINS

The mixing chamber at the lowest part of the refrigerator provides the necessary cooling power to bring the processor and associated components down to a temperature of 15 mK – colder than outer space.

15 MILLIKELVINS

Cryogenic isolators enable qubit signals to go forward while preventing noise from compromising qubit quality.

The quantum processor sits inside a shield that protects it from electromagnetic radiation in order to preserve its quality.

SUPERCONDUCTING COAXIAL LINES

CRYOGENIC ISOLATORS

MIXING CHAMBER

QUANTUM AMPLIFIERS

CRYOPERM SHIELD



**We are close to a universal quantum computer, here's where we are at**



## ❖ Modeling large complex systems

- the brain
- the universe
  - Can describe an atom with a few bits
  - Takes 100 bits to describe atoms interacting
  - $2^{100}$  or  $10^{100}$  bits,  $10^{90}$  particles in whole universe
  - Few 100 qubits easily solves this problem
- physics or chemistry





# The future of quantum computing

---

- ❖ **In 100 years quantum computers will be boring**
  - Maybe not in all households, but common
  - Molecular computers in households?
- ❖ **Think up more problems to solve...won't be able to solve them quite yet**



# End of Lecture 7

Have you realized the power of swarm intelligence and known how to simulate it?

LIUXIABI@BIT.EDU.CN

